

大语言模型本地部署与优化

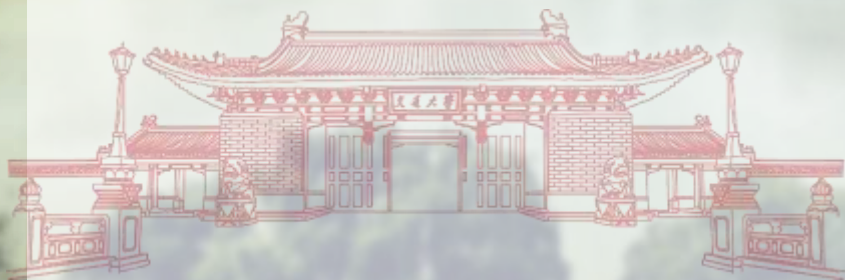
学生创新中心

2025年12月

饮水思源 · 爱国荣校

01

大语言模型介绍



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

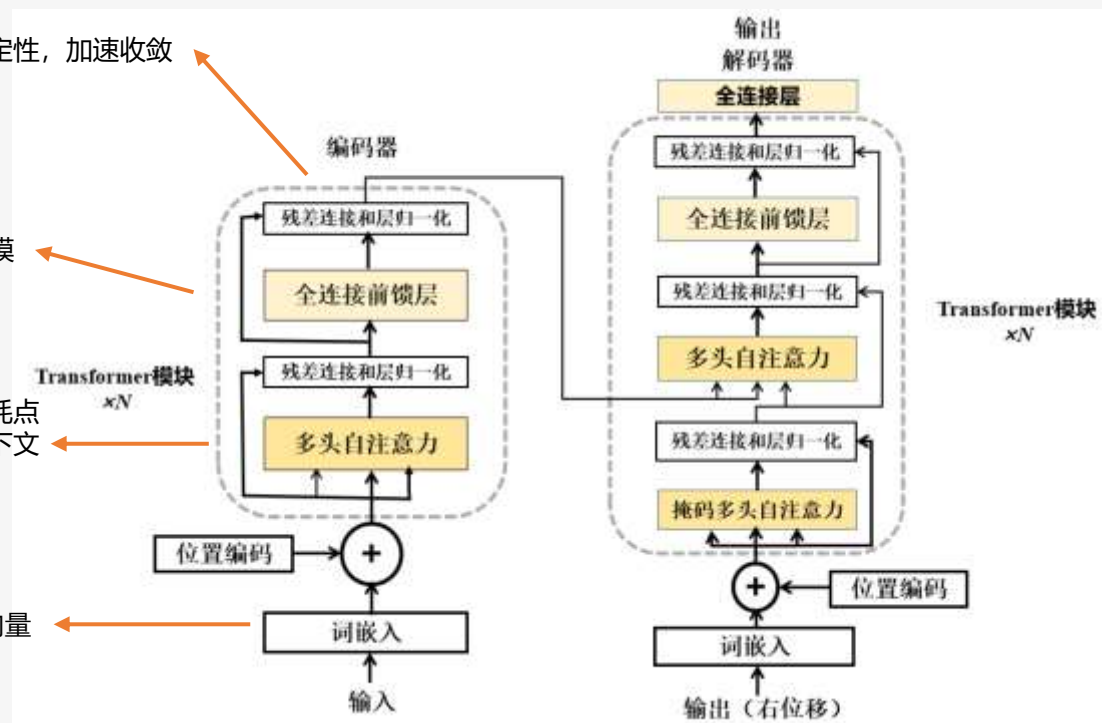
[‡]Work performed while at Google Research.

- 提高训练稳定性，加速收敛

- 参数集中地
- 负责存储大模型知识

- 推理时间消耗点
- 负责联系上下文

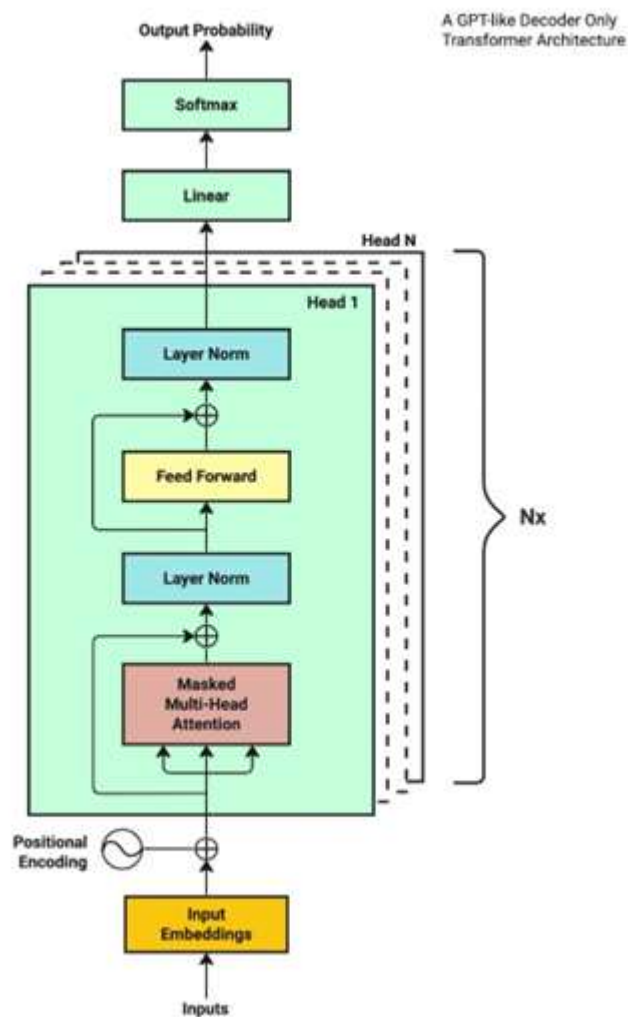
- Token转成向量



GPT架构图



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



- 起步最早
- 逐渐闭源
- 做大模型



- 起步较晚
- 坚持开源
- 提高效率

模型	发布时间	参数量 (亿)	语料规模
GPT-1	2018.06	1.17	约5GB
GPT-2	2019.02	1.24/3.55/7.74/15	40GB
GPT-3	2020.05	1.25/3.5/7.62/13/27/67/130/1750	1TB
LLaMA-1	2023.02	67/130/325/652	约5TB
LLaMA-2	2023.07	70/130/340/700	约7TB
LLaMA-3	2024.04	80/700	约50TB



大模型的发展历程

技术积累阶段

大规模验证时期

迈向Agent 时代

2015~2020年

2021~2024年

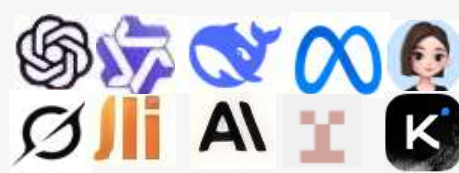
2024~至今

谷歌、微软、百度



数据挖掘（图谱分析、few shots）

百模大战



Coze平台、Cursor、微软
365 copilot、Manus等



A: Transformer架构演进
B: 大模型语料积累
C: CUDA生态及Nvlink技术

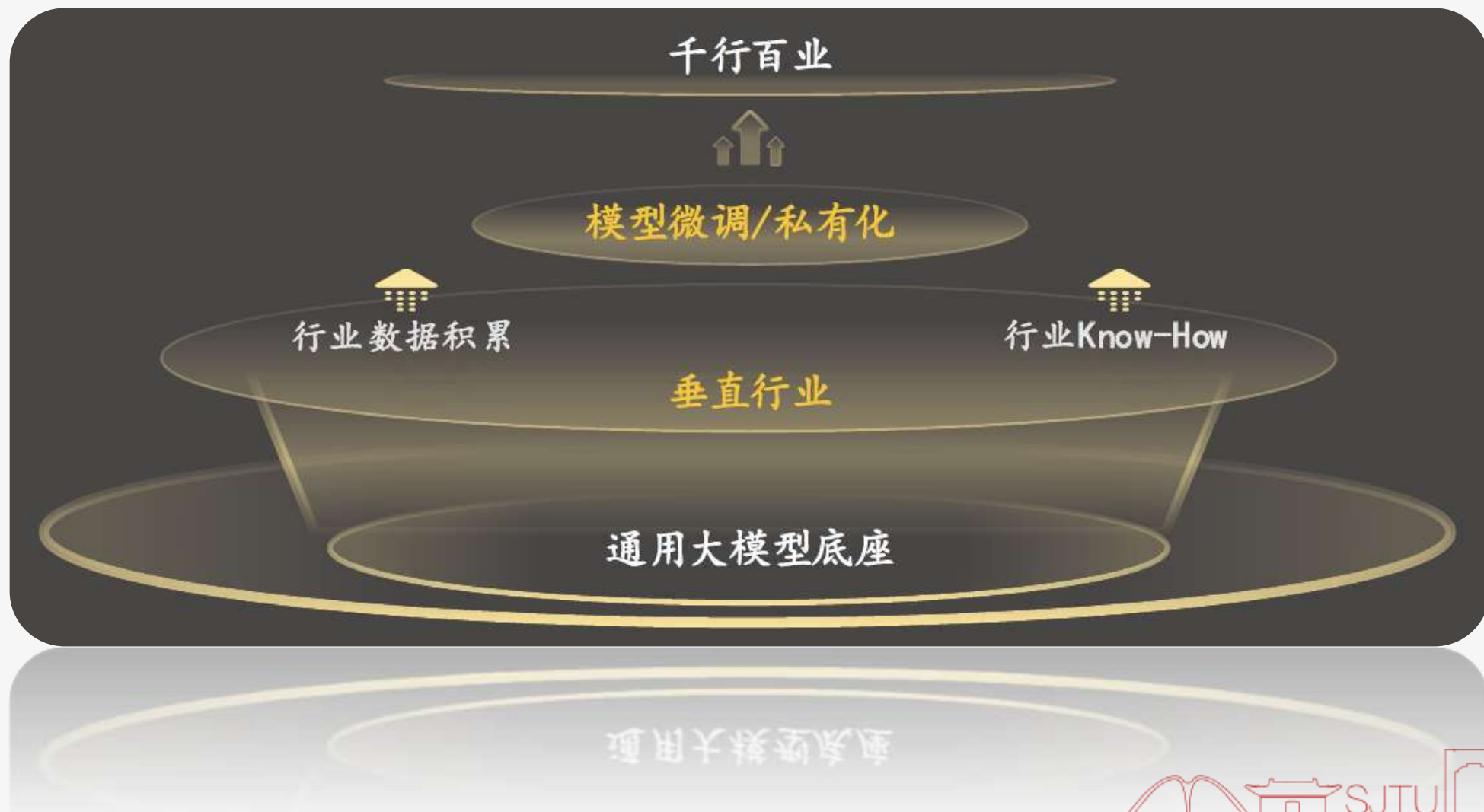
GPT-3: Decoder only架构
Chat-GPT: RLHF指令微调

大模型智能提升速度开始收敛
开始产生商业价值

“大模型”走进千行百业

行业落地面临的问题

- 准确性问题
- 安全性问题
- 性价比



敏感数据的隐私保护

医疗、金融、法律、政府等领域，符合隐私保护、数据安全、杜绝泄密隐患。

成本控制

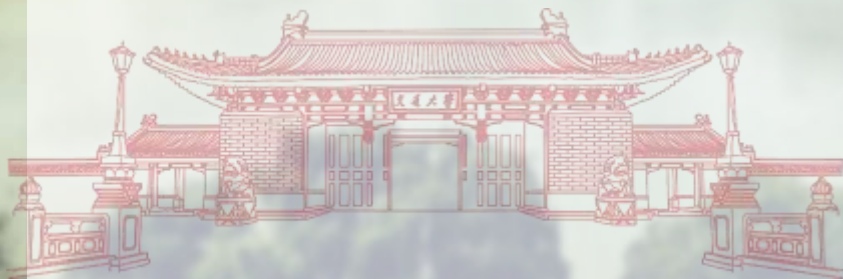
云服务按使用量收费，初期成本低但长期费用高；本地部署一次性投入硬件与软件，长期成本低。

更为灵活

本地部署可对模型进行微调，满足企业特定业务需求，提升模型性能与准确性。

02

语言模型 本地部署实战



LLM 服务框架概览



开源 LLM 服务框架能够高效地部署和提供服务。如：Ollama、vLLM、Llama.cpp、xinference等。

Ollama: 专为简化 LLM 在本地运行而设计的开源框架，它提供一个轻量级服务器，能够轻松地下载、管理和运行各类 LLM 模型，设置过程简单易用。它支持 4 位或 5 位模型量化（通常采用 GGUF 格式），Ollama 通常一次为每个模型处理一个请求，批处理能力有限，不适用于大规模并发服务场景。

vLLM: 高性能 LLM 推理的 GPU 加速器，专注于在 GPU 服务器上最大化模型推理的吞吐量和效率，提供与 OpenAI 兼容的 HTTP API，vLLM 的核心在于通过创新的内存管理和调度算法，有效解决了传统 Transformer 模型推理中的性能瓶颈。适合追求极致推理效率和可扩展性的场景。

```
os.environ["no_proxy"] = 'xxx'
BASE_URL = "http://192.168.1.137:8000/v1"
API_KEY = "EMPTY"
MODEL_CHAT = "Qwen3-30B-A3B"

client_chat = OpenAI(base_url=BASE_URL, api_key=API_KEY)

resp = client_chat.chat.completions.create(
    model=MODEL_CHAT,
    messages=[{"role": "user", "content": "请给出一个交大闵行校区周边一日游旅行计划"}],
    extra_body={"chat_template_kwargs": {"enable_thinking": False}}, # 关闭 thinking 功能
    temperature=0,
    max_tokens=4096
)
print(resp.choices[0].message.content)
```

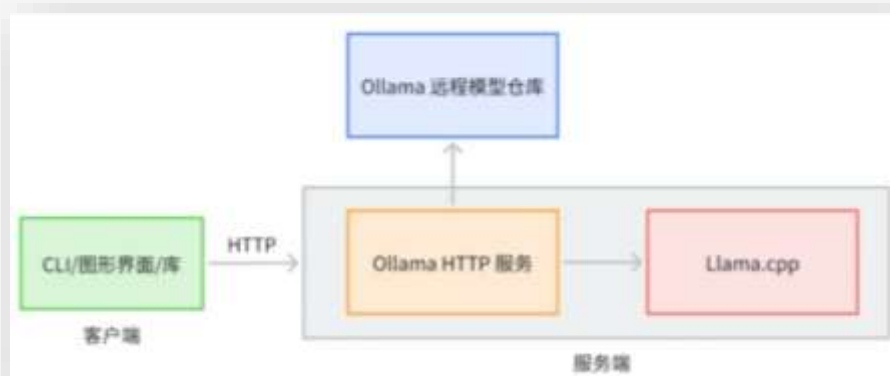


Ollama 与vLLM性能对比

维度	Ollama	vLLM
使用场景	面向个人用户，适合本地部署和离线推理	更适用于高并发、生产级推理服务场景
硬件要求	支持 CPU/AppleSilicon/各类 GPU	主要针对高性能 GPU 优化(Nvidia GPU + CUDA)
是否支持纯CPU	支持，具备针对指令集的优化	仅部分实验性支持，性能有限
吞吐性能	更偏向单请求场景批处理能力有限	吞吐高，支持分页注意力连续批处理、PD分离等机制
分布式推理	仅支持单机运行	支持跨节点的分布式推理
使用难度	安装简单，开箱即用	配置较复杂，需具备Python 基础及一定LLM部署经验
OS支持	支持 Windows /macOS /Linux	当前仅支持 Linux

Ollama的安装与使用

Ollama 采用Client-Server 架构:客户端↔ 服务端(HTTP 服务 + 推理引擎)



支持的模型

- LLaMA (Meta 的开源模型)
- Qwen
- Deepseek
-

1 客户端：与 AI 交互的“界面”

可以通过多种方式与 Ollama 交互：

- 🖥️ 命令行（如：ollama run llama3）
- 🌐 图形界面（例如 Open WebUI、Ollama Desktop）
- 💻 程序接口（Python/JS 库，适合开发集成）

2 服务端（HTTP 服务）：本地运行的“调度中心”

Ollama 服务端是一个在后台常驻进程，协调整体运行流程：

- 👉 接收客户端的请求
- 👉 管理模型缓存（判断是否已下载、是否需要更新）
- 👉 调用推理引擎生成 token
- 👉 将生成结果返回客户端

Ollama的安装与使用

Ollama常用命令:

```
ollama run qwen2.5:latest
```

1.创建模型

- ollama create [Modelfile路径]: 使用包含模型信息的Modelfile来创建新模型。

2.显示模型信息

- ollama show [模型名称]: 显示特定模型的详细信息, 如模型名称、版本等。

3.列出模型

- ollama list: 列出本地所有可用的模型

4.运行模型

- ollama run [模型名称]: 运行一个已安装的模型, 执行某些任务。

5.删除模型

- ollama rm [模型名称]: 删除一个已安装的模型。

Ollama模型库:

<http://ollama.com/library>



The screenshot shows the Ollama model library page for the 'qwen2.5' model. It includes a search bar with 'ollama run qwen2.5', a download count of 11.8M, and a description stating the model is pretrained on Alibaba's latest large-scale dataset. Below this is a table of available models.

Name	Size	Context	Input
qwen2.5:latest	4.7GB	32K	Text
qwen2.5:0.5b	389MB	32K	Text
qwen2.5:1.5b	966MB	32K	Text
qwen2.5:3b	1.8GB	32K	Text
qwen2.5:7b	4.7GB	32K	Text
qwen2.5:14b	9.0GB	32K	Text
qwen2.5:32b	28.0GB	32K	Text
qwen2.5:72b	47GB	32K	Text

Ollama API调用

```
import ollama

# 流式调用 DeepSeek 模型
def deepseek_stream(prompt):
    response = ollama.generate(
        model="deepseek-r1:1.5b", # 模型名称
        prompt=prompt,           # 输入提示
        stream=True,              # 启用流式输出
    )
    for chunk in response:
        yield chunk["response"] # 逐块返回生成结果
```

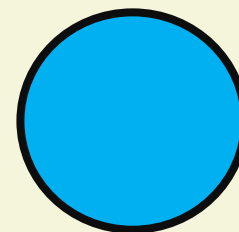
Ollama client server 调用模式

```
from ollama import Client
```

```
client = Client(host="http://服务器ip:11434")
```

```
client.generate(model="qwen2.5:latest", prompt="你好")
```

动手实践时间



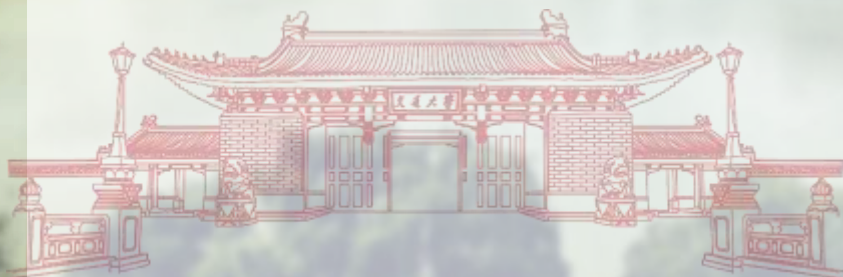
Demo Time!

Ollama+Qwen2.5本地部署



03

Gradio介绍



Gradio是一个用于支持快速构建可视化网页demo应用的Python框架。

特点:

- python代码集成, 快速搭建前端原型, 方便使用;
- 创建链接即可实现外部访问;
- 无需JavaScript, CSS相关的web开发经验;



Fork 2.1k



Star 28.6k



课前准备 - 环境安装

1.python环境安装

- 建议使用anaconda管理python环境
- anaconda下载:
- 要求: Python 3.8 +
- conda环境安装命令:
 - `conda create -n {gradio_env_name} python=3.10`

2. Gradio安装

版本: 4.26.0

安装命令: `pip install gradio --i https://pypi.tuna.tsinghua.edu.cn/simple/`

3.进入conda环境

命令: `conda activate {gradio_env_name}`

Try your first gradio demo

Code:

```
01.demo_start.py > ...  
1  import gradio as gr  
2  
3  def greet(name, intensity):  
4      return "Hello " * intensity + name + "!"  
5  
6  demo = gr.Interface(  
7      fn=greet,  
8      inputs=["text", "slider"],  
9      outputs=["text"],  
10 )  
11  
12 demo.launch()
```

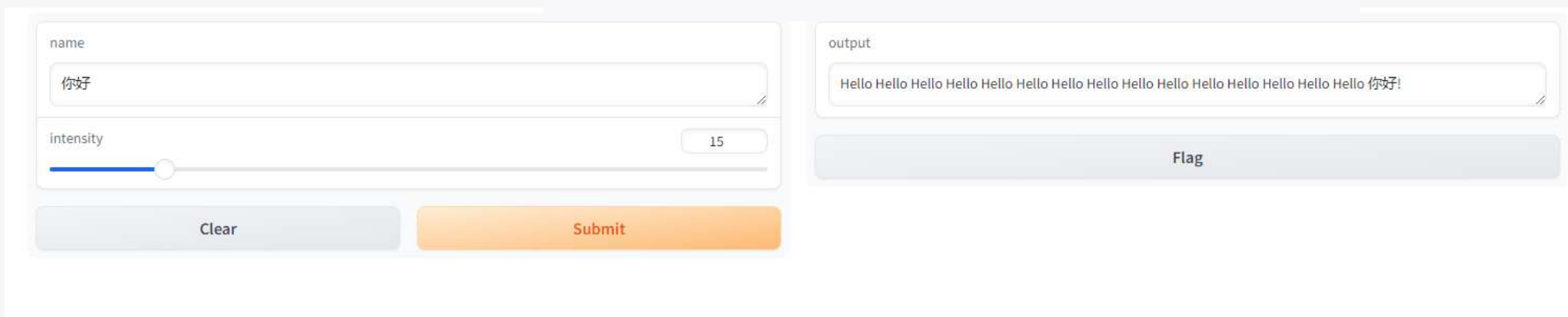
Running logs:

```
Running on local URL:  http://127.0.0.1:7860
```

```
To create a public link, set `share=True` in `launch()`.
```

- **fn** 是处理逻辑的函数。
- **inputs** 和 **outputs** 定义处理函数的输入和输出类型。

Visualization
interface:



The visualization shows a web interface with two input sections on the left and one output section on the right. The first input section, labeled 'name', contains a text box with the value '你好'. The second input section, labeled 'intensity', contains a slider with a value of 15. Below these inputs are two buttons: 'Clear' and 'Submit'. The output section, labeled 'output', contains a text box with the value 'Hello Hello Hello Hello Hello Hello Hello Hello Hello Hello Hello Hello Hello Hello 你好!'. Below the output text box is a button labeled 'Flag'.

Gradio快速搭建 - interface

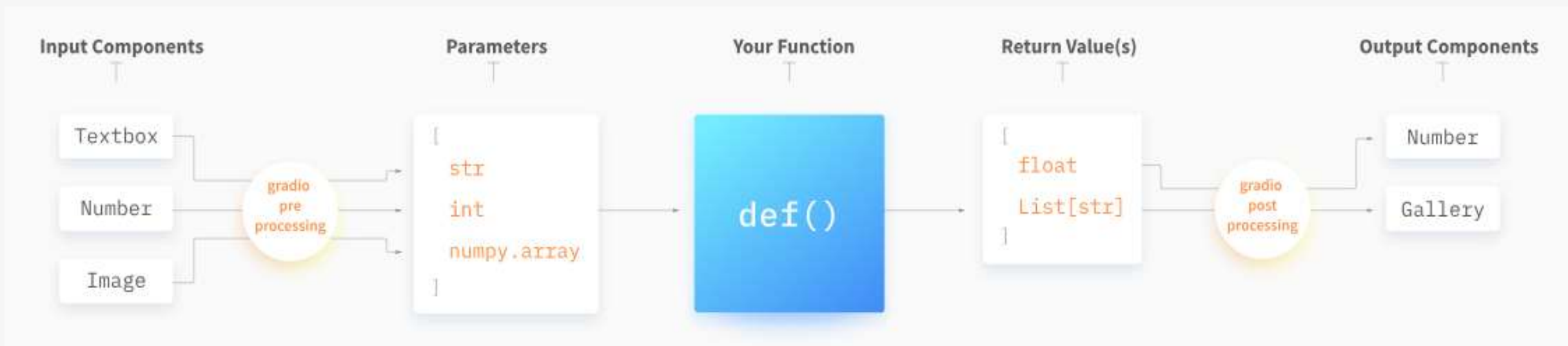
使用gradio.Interface快速搭建 GUI demo, 三个required 参数:

- fn: 需要封装的接口函数。这通常是一个机器学习模型的预测函数;
- Inputs: 一个单独的Gradio组件, 或者多个Gradio组件构成的列表;
- Outputs: 与inputs要求相同;

Gradio运行流程

Gradio运行流程包括：

- 输入预处理
- 函数调用
- 返回输出
- 结果后处理



Gradio包含30多个预先构建的组件（支持用户自定义组件），用于创建交互式界面，这些组件这些组件对应于机器学习中常见数据类型：

应用界面：

gr.Interface(简易场景), gr.Blocks(定制化场景)
gr.Chatinterface(对话场景)

布局组件：

gr.Tab(标签页),
gr.Row(行布局), gr.Column(列布局)
gr.Group()
cols : 指定输入和输出组件的列数
rows : 指定输入和输出组件的行数

输入输出：

gr.Image(图像), gr.Textbox(文本框), gr.DataFrame(数据框),
gr.Dropdown(下拉选项), gr.Number(数字),
gr.Button(按钮) gr.Checkbox：用于创建复选框。
gr.Radio：用于创建单选按钮组。
gr.Slider：用于创建滑块。
gr.ColorPicker：用于颜色选择。
gr.Markdown：用于Markdown输入和输出。
gr.Files：用于文件输入和输出。
gr.Plot：用于制作图表。
gr.HTML：用于嵌入HTML内容。
gr.Audio：用于音频输入和输出。
gr.Video：用于视频输入和输出。
gr.Label: 用于显示分类标签。

Share to others

实现公网(非同一个局域网)访问gradio

Running on local URL: `http://0.0.0.0:9103`

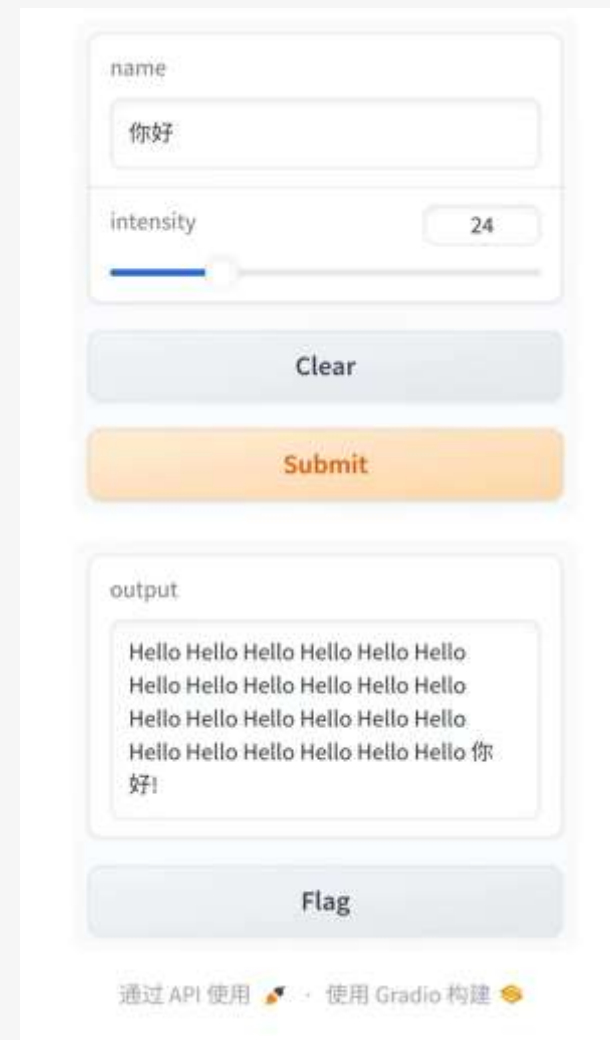
Running on public URL: `https://55c345795221660a98.gradio.live`

Access generate address

```
demo.launch(share=True, server_port=9103)
```

Access ip+port

```
demo.launch(server_name="0.0.0.0", share=True, server_port=9103)
```



The image shows a Gradio web interface. At the top, there is a text input field labeled 'name' containing the text '你好'. Below it is a slider labeled 'intensity' with a value of 24. There are two buttons: 'Clear' and 'Submit'. Below the buttons is an 'output' section displaying the text 'Hello Hello Hello Hello Hello Hello Hello Hello Hello Hello Hello 你好!'. At the bottom, there is a 'Flag' button and a footer with the text '通过 API 使用' and '使用 Gradio 构建'.

Problems occurred when create share link

```
Running on local URL: http://0.0.0.0:9103
```

```
Could not create share link. Missing file: /home/lighthouse/conda/envs/gradio_env/lib/python3.10/site-packages/gradio/frpc_linux_amd64_v0.2.
```

```
Please check your internet connection. This can happen if your antivirus software blocks the download of this file. You can install manually by following these steps:
```

1. Download this file: https://cdn-media.huggingface.co/frpc-gradio-0.2/frpc_linux_amd64
2. Rename the downloaded file to: `frpc_linux_amd64_v0.2`
3. Move the file to this location: `/home/lighthouse/conda/envs/gradio_env/lib/python3.10/site-packages/gradio`

根据提示解决该问题的步骤：

- 下载 frpc 文件
- 对文件进行重命名
- 移动文件到conda环境对应的gradio文件夹下

```
Running on local URL: http://0.0.0.0:9103
```

```
Running on public URL: https://55c345795221660a98.gradio.live
```

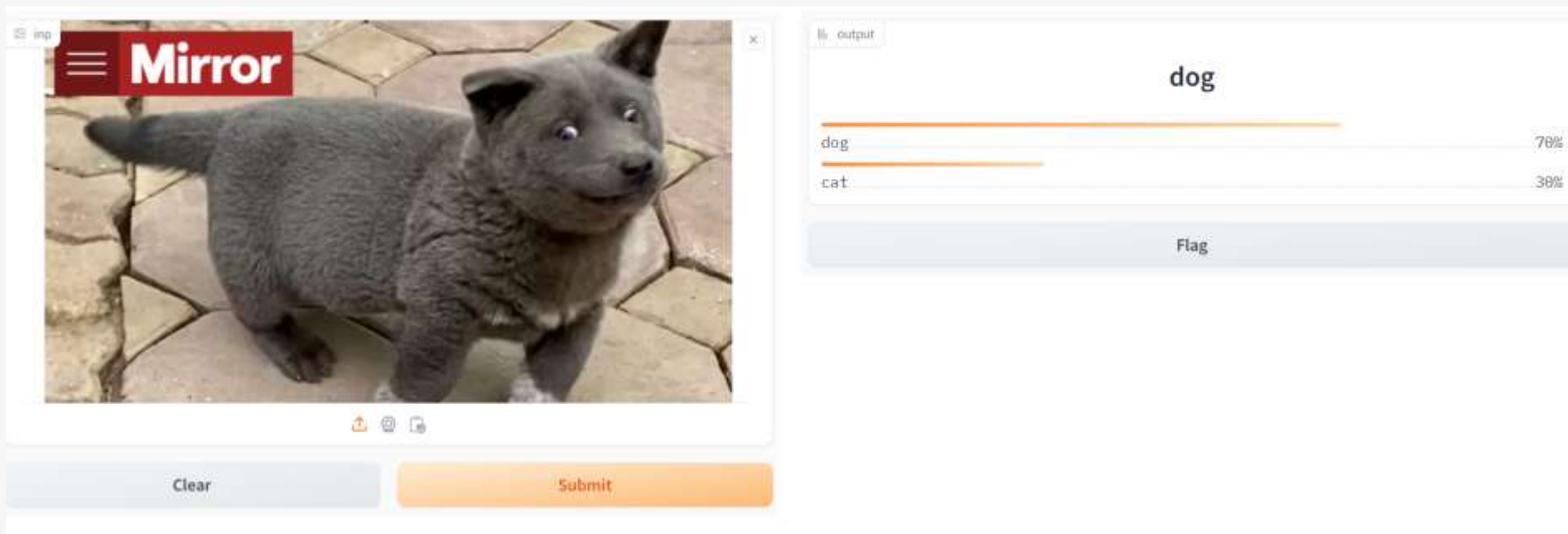
```
This share link expires in 72 hours. For free permanent hosting and GPU upgrades, run `gradio deploy` from Terminal to deploy to Spaces (https://huggingface.co/spaces)
```

Gradio快速搭建 – interface demo

```
import gradio as gr

def image_classifier(inp):
    return {'cat': 0.3, 'dog': 0.7}

demo = gr.Interface(fn=image_classifier, inputs="image", outputs="label")
demo.launch()
```



Gradio快速搭建 – interface 输入控制组件

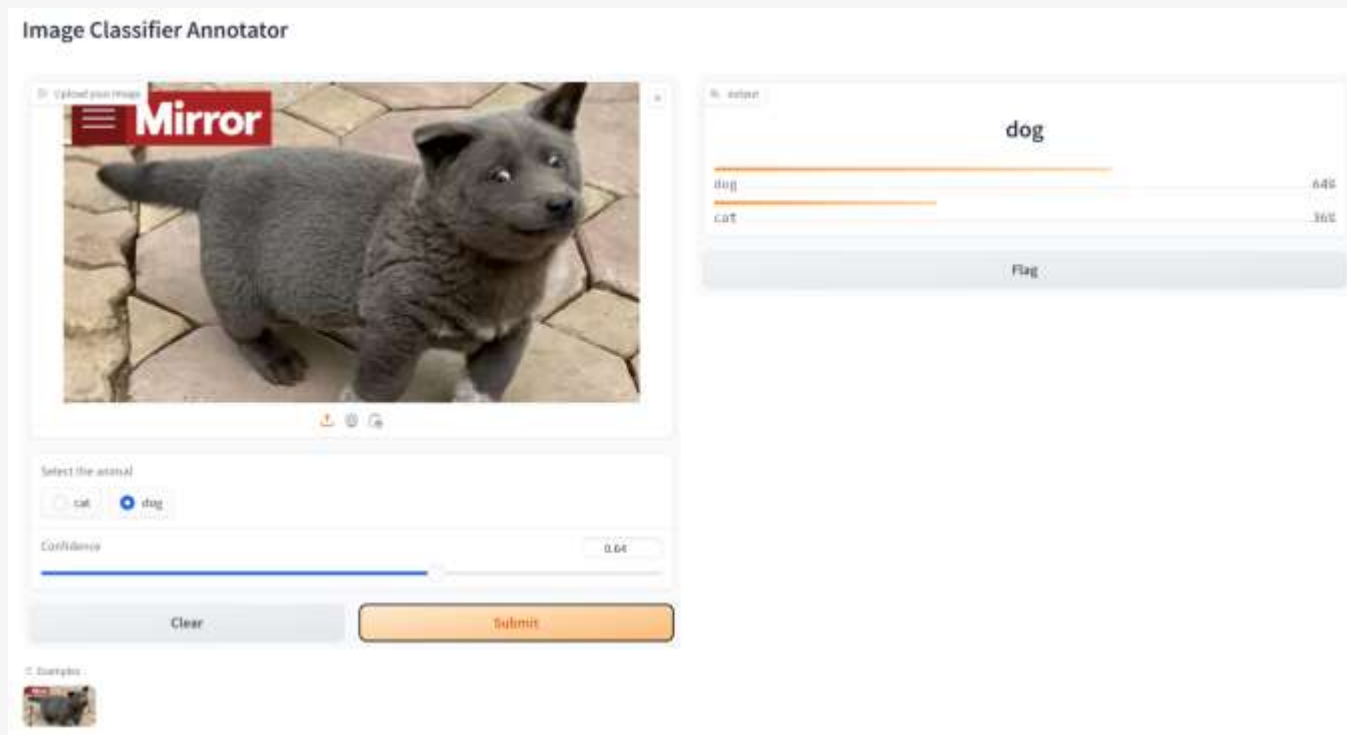
通过更多gradio的输入控制组件实现更多样化的需求，以实现一个简单的图像分类标注前端为例。

```
import gradio as gr

cls_name_list = ['cat', 'dog']

def image_classifier(inp, cls_name, cls_confidence):
    if cls_name == 'cat':
        return {f'{cls_name}': cls_confidence, 'dog': 1-cls_confidence}
    else:
        return {f'{cls_name}': cls_confidence, 'cat': 1-cls_confidence}

demo = gr.Interface(fn=image_classifier,
                    inputs=[
                        gr.Image(label="Upload your image"),
                        gr.Radio(["cat", "dog"], label="Select the animal"),
                        gr.Slider(0, 1, 0.5, label="Confidence")
                    ],
                    outputs="label",
                    title="Image Classifier annoator",
                    examples=[["./data/dog.png"]])
demo.launch()
```



Gradio-对话interface

gradio.ChatInterface是用于创建chatbot对话UI的高级接口，只需要将对话模型的封装函数和输入作为参数传入，就能搭建常见的对话体验界面。

fn 函数必须接受 两个参数，按顺序依次为：

message：当前用户发送的字符串消息（即用户最新输入的内容）。

history：包含整个对话历史的列表，格式为 [(用户消息1, 机器人回复1), (用户消息2, 机器人回复2), ...]。

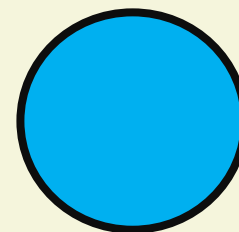
```
import gradio as gr

def echo(message, history):
    return message["text"]

demo = gr.ChatInterface(fn=echo, examples=[{"text": "hello"},
                                           {"text": "hola"},
                                           {"text": "merhaba"}],
                        title="Echo Bot", multimodal=True)
demo.launch()
```



动手实践时间



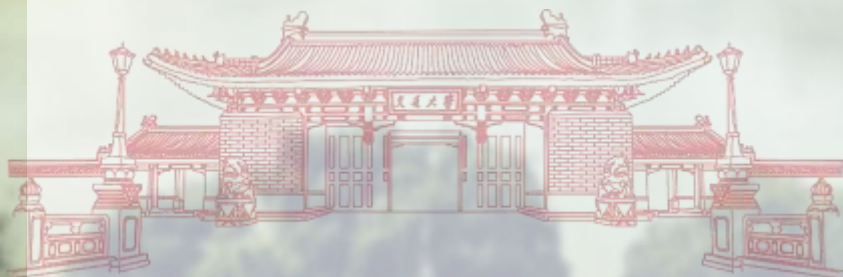
Demo Time!

2. Gradio搭建简单app



04

RAG (检索增强生成)



为什么引入RAG?

LLM的缺陷:

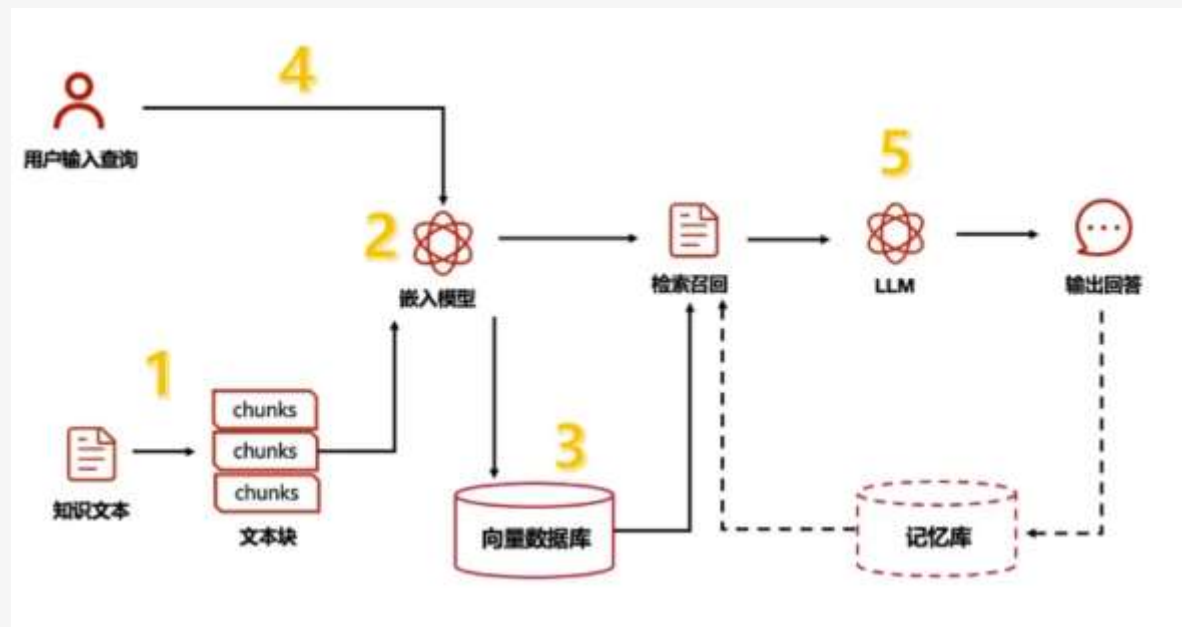
- 幻觉 (Hallucination) : LLM 可能生成无事实依据、不与现实世界一致, 甚至完全虚构的内容。
- 知识更新滞后 (Knowledge Cut-off) LLM知识的有效性取决于训练数据的时间, 因此存在知识更新滞后的问题。
- 回答缺乏透明度 (Lack of Transparency in Answers) LLM 生成的回答通常缺乏引用来源, 加上幻觉问题, 导致用户难以判断答案的真实性, 降低了模型的可信度。

解决方法之一: **RAG (Retrieval-Augmented Generation)** 检索增强生成, RAG相当于外挂知识库, 形成更完整的输入提示。用于弥补大模型某些知识的欠缺, 从而提升回答问题的准确性。

什么是RAG?

RAG技术结合**检索**、**增强**、**生成**三个步骤。首先，检索相关文档或库，找到最相关的片段作为证据。然后，结合用户输入一起送给大模型辅助生成精确的答案。

特性	RAG优势
事实准确性	通过外部检索，减少幻觉问题
知识更新	依赖实时数据，无需重新训练模型
可解释性	生成结果带有检索来源，提高可信度
数据可控性	可使用特定数据源，如企业知识库



RAG的应用场景

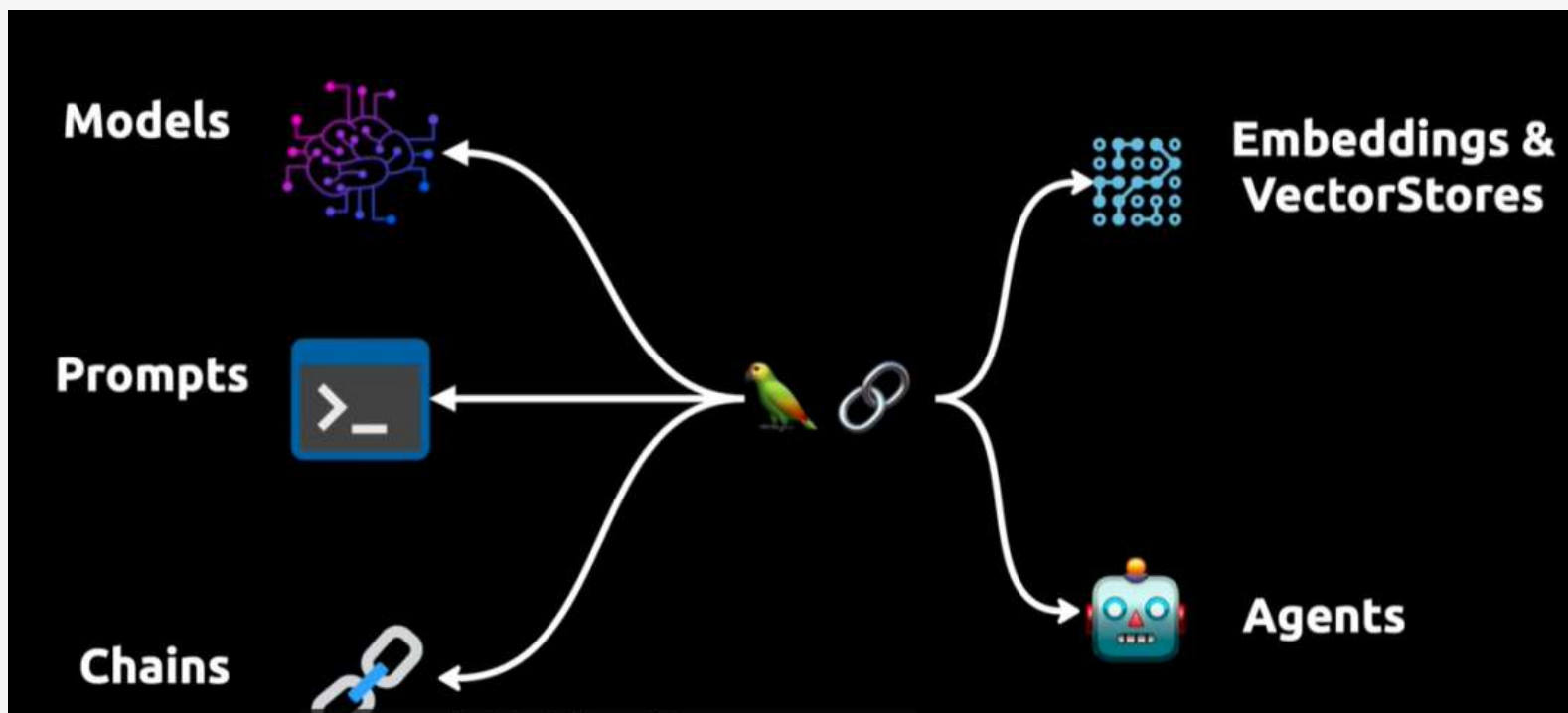
- ✓ **企业知识问答** (Enterprise Q&A) :
 - 结合企业内部文档, 实现精准问答
- ✓ **法律/医疗领域问答**:
 - 结合权威法律法规、医学文献, 提高回答准确性
 - 例如 LexisNexis AI、医学 AI 诊断
- ✓ **金融风控**:
 - 结合市场数据和金融法规, 辅助投资分析
 - 例如 BloombergGPT
- ✓ **搜索增强对话** (Chatbot Augmentation) :
 - 结合搜索引擎和 LLM, 提高问答质量
 - 例如: Perplexity AI、ChatGPT Bing

RAG的技术路线

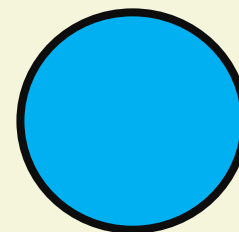
1. **用户查询** (User Query) : 用户输入问题。
2. **文档切分** (Document Chunking) : 将知识库中的文档分成小段。Overlap、长度、对结果影响比较大。
3. **向量化**: Embedding, 比较耗时。
4. **向量检索** (Vector Retrieval) : 使用向量数据库 (如 FAISS、Pinecone) 检索相关文档。
5. **相关文档匹配** (Related Document Chunks) : 获取最相关的文本片段。
6. **Prompt 构造**: 将检索到的文本与用户查询一起输入 LLM。
例如: “请根据找出的{context}知识相关内容回答{question}”
7. **LLM 生成回答**。

什么是LangChain?

- 用于开发LLM应用的开源框架，提供复杂的工具和API接口调用。轻松地将大语言模型集成到应用程序中，处理语言模型生成的响应、对话管理、链式调用等。
- 组件众多：为使用语言模型提供抽象层以及抽象层的实现。组件是模块化且易于使用的。包含实现RAG、Agent的各种工具。
- 现成的链：结构化的组件集合，针对常规的应用流程，利用链(Chain) 完成特定的高级任务。



动手实践时间



Demo Time!

3. Langchain+RAG



Demo1: 从pdf中获得检索信息



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

LangChain_community是 LangChain框架中由开发者社区维护的第三方集成库，主要用于扩展 LangChain的核心功能，使其能够与数百种外部服务和工具无缝对接。所有集成代码由全球开发者共同维护，LangChain 团队仅提供标准接口规范。这种模式既保证了功能的多样性，又通过社区审核机制确保代码质量。

知识文档的准备：知识源可能包含多种格式，例如：Word文档、TXT文件、CSV数据表、Excel表格，甚至是PDF文件、图片等，可以使用专门的文档加载器，将这些丰富的知识源转换为大语言模型可理解的纯文本数据。document_loaders 返回包含元数据的Document对象。

文本切片：鉴于文档可能存在过长的问题，需要将长篇文档分割成多个文本块，以便更高效地处理和检索信息。这不仅有助于减轻模型的负担，还能提高信息检索的准确性。

嵌入模型：将文本转换为向量形式，并且更加密集、精确地捕捉到句子的上下文关系和核心含义。

向量数据库：通过嵌入模型生成的所有向量都会被存储在这样的数据库中，专门设计用于存储和检索向量数据，优化处理和存储大规模向量数据的效率。

查询检索：用户的问题会被输入到嵌入模型中进行向量化处理。然后，系统会在向量数据库中搜索与该问题向量语义上相似的知识文本或历史对话记录并返回。



结果对比

```
query = "transformer快速入门应该如何学习?"
response = qa_chain.run(query)
print(response)
```

6m 41.7s Python

<think>
好的，我现在要回答用户的问题：“transformer快速入门应该如何学习？”首先，我需要理解用户的问题。他们可能是一个对大语言模型（LLM）感兴趣或者正在做项目的
接下来，我要分析这些资源中的信息。从提供的13条内容来看，每个链接都提到了学习的大致方向，比如“整理开源的中文大语言模型”，“练习这些项目”，“快速入门教程”
然后，我注意到有一个链接指向“https://datawhalechina.github.io/so-large-lm/#/content/ch14”，这是大模型理论基础的资料之一。虽然这不是具体的入门教程，
还有一个重要的资源是第13条内容中的仓库链接，“https://github.com/HoHu-HITCS/Awesome-Chinese-LLM”，这是一个名为“Awesome-Chinese-LLM”的仓库，可能
接下来，我需要考虑如何组织这些信息来回答问题。首先，可以提到Llama社区作为一个强大的学习工具，特别是在中文领域。然后，推荐一些具体的项目或仓库，如第8条
还要确保回答清晰明了，使用简单的语言，避免使用复杂的术语，让用户容易理解。如果有遗漏的信息或需要补充的内容，请尽量指出，但根据现有信息，我能够提供一
</think>

为了快速入门大模型的“transformer”技术，可以从以下几个方面入手：

1. ****学习Llama社区****：Llama中文社区是一个强大的资源库，专注于中文的大语言模型开发。建议访问该仓库以获取大量中文教程和项目。
2. ****阅读开发者项目的指南****：参考第8条中的开发者项目——“人工智能指导与示例”，这是一个针对小白开发者的入门教程，详细介绍了大模型的应用和开发流程。
3. ****使用开源项目****：推荐阅读第13条中的“Awesome-Chinese-LLM”仓库。这是一个中文版本的大模型资源库。通过使用该资源，可以进行代码实践，加深理解。
4. ****学习九层妖塔模型****：参考第6条提供的“llm-course”项目。这是一个专注于九层妖塔大模型的入门指南，提供了详细的步骤和框架。
5. ****深入理论基础****：阅读第7条中的大模型知识面试题资源，了解理论基础上的大模型技术原理，并结合实际应用实践。

通过这些资源的学习和实践，你可以逐步掌握“transformer”技术的基础知识和应用能力。

学习注意力机制是个重点。我之前学过机器学习中的卷积神经网络，但现在要深入理解自注意力如何改变模型的结构，可能需要花时间研究自注意力计算、矩阵运算以及注意力权重的计算方法。

然后显示序列处理部分。Transformer如何处理序列数据？我需要了解输入序列是如何被编码为向量的，比如词 embedding，这可能是通过预训练模型得到的。接着，解码器的部分怎么进行呢？它是不是按照时间顺序生成每个单词？

还有，Transformer的整体架构，包括编码器和解码器，以及多注意力机制的应用。我需要理解它们是如何协同工作的，可能还需要看一些示例代码或者实验结果。

接下来是实际应用部分。我可以尝试用一个简单的模型来做个小项目，比如生成Hello World或者做一些简单的翻译任务。这些实验可以加深我的理解和练习我的编程能力。

最后，我要关注最新的进展。Transformer在NLP中的应用越来越广泛，比如多模态模型、序列到图像等技术。了解这些可能会让我更全面地掌握Transformers的应用方向。

学习资源方面，推荐一些书和在线课程。深度学习的书籍，像《Deep Learning》或者《Neural Networks and Deep Learning》。还有相关论文和教程，比如“Attention is All You Need”这篇经典论文，以及使用TensorFlow或PyTorch的入门教程。

总结一下，我的计划是：

1. 学习Transformer的基本原理和工作机制。
2. 研究自注意力机制的数学细节。
3. 理解编码器和解码器的结构及其作用。
4. 掌握基本的数据预处理步骤。
5. 开始实践一个小模型，如Hello World。
6. 关注最新的研究进展，了解多模态学习和生成模型的发展。

在这个过程中，可能会遇到一些困难，比如数学基础不够扎实会影响理解。如果时间允许的话，可以考虑寻求导师或参加在线课程的补充部分。同时，保持耐心，逐步深入，不要急于求成。这样可以更好地掌握Transformers的基础知识。

</think>

要快速入门学习Transformer模型，可以从以下几个步骤进行：

1. ****了解基本概念和目标****：学习Transformer的核心思想及其在NLP中的应用。
2. ****学习序列模型基础****：熟悉RNN、LSTM等传统 seq model的工作原理。
3. ****深入了解自注意力机制****：研究自注意力如何改变模型结构，掌握矩阵运算和权重计算。
4. ****探索编码器与解码器的结构****：理解它们如何将输入转换为向量并生成输出。
5. ****实践数据预处理****：使用CommonCrawl或GPT-2离线数据集，进行文本分词和嵌入训练。
6. ****实现模型小项目****：例如Hello World翻译，验证学习成果。
7. ****关注最新进展****：了解多模态技术及其在生成任务中的应用。

通过系统的学习与实践，逐步掌握Transformer的原理及应用。



Demo2: 从web端获得检索信息



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

WebBaseLoader: 将网页数据源转换为大语言模型可理解的纯文本数据。

FAISS: **高效向量索引构建**, 通过索引优化降低搜索复杂度, 向量压缩, 减少向量存储空间。

结合Gradio以及Ollama本地部署 deepseek, 可以轻松实现基于网页的RAG问答应用。

RAG链网页内容问答

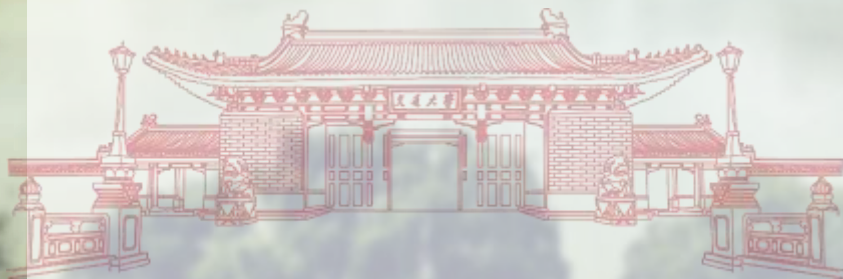
输入URL并从RAG链中获取答案。

url	output
https://www.sjtu.edu.cn/content/person-detail?-----60202-f757-f4f0b555-04b-d1f430408-----	<think> 嗯, 我现在需要回答的问题是: “楚朋志的专业是? ”。首先, 我要找出提到“楚朋志”的信息。 在提供的材料中, 我看到有一行文字: “公益服务”和“用户登录”, 没有涉及到这个名字。接下来, 阅读后面的段落, 发现有一个地方提到了“人工智能实践课程”。具体来说, 它是在实验室研究与探索的论文中提到的: 人工智能实践课程, 32学时/2学分。这里的“人工智能”显然是楚朋志的专业。
question	
楚朋志的专业是?	
Clear	Submit
Flag	



05

Function calling

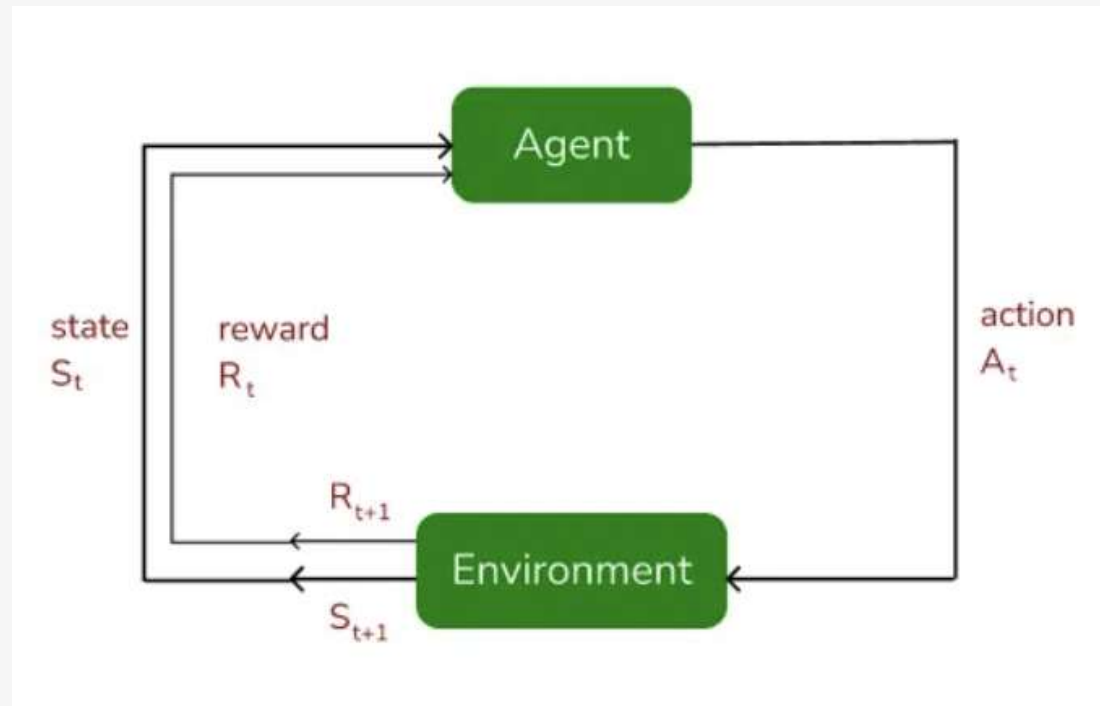


什么是Agent?

智能体(Agent)是强化学习系统中的学习者和决策者。它感知环境状态, 并根据其策略 (Policy) 选择行动。

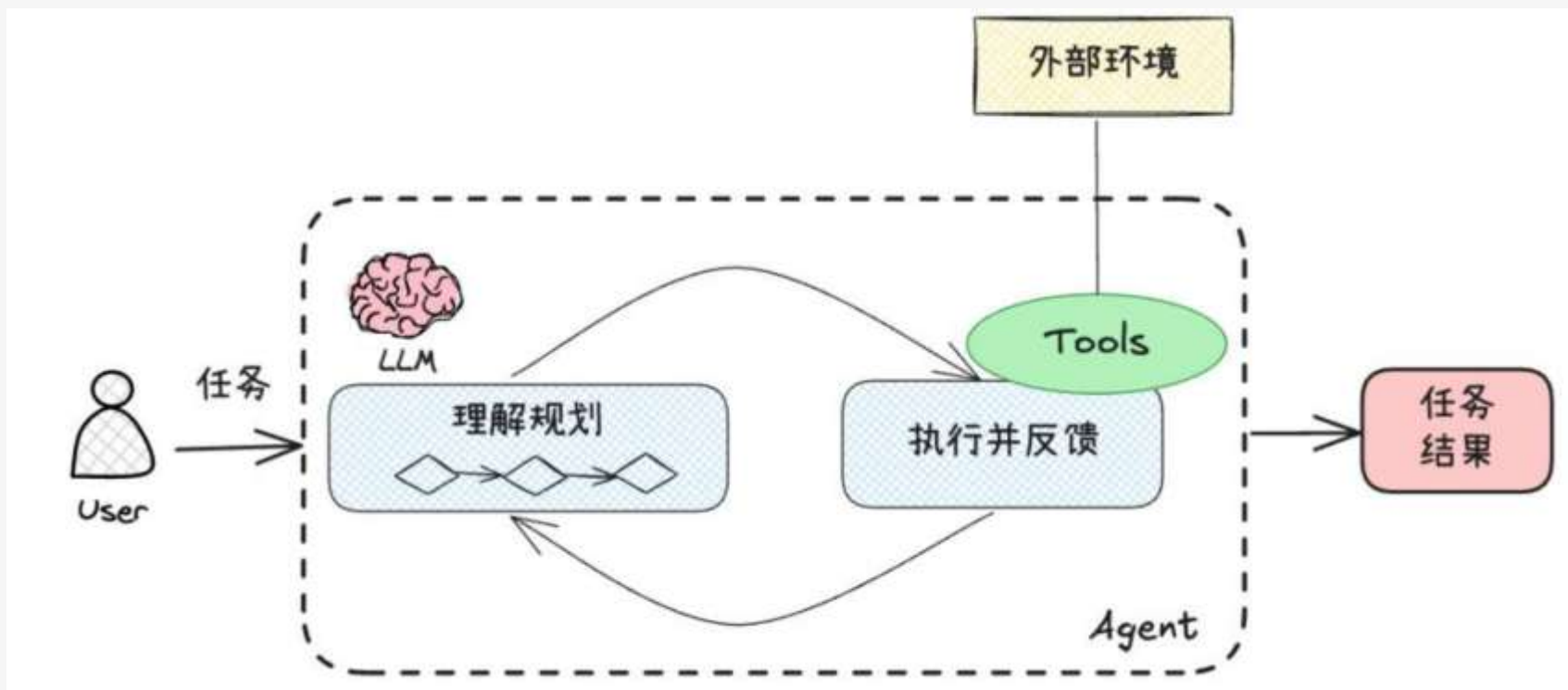
核心职责:

- **1. 感知:** 接收环境反馈的状态 (State) 和奖励 (Reward) 。
- **2. 规划:** 根据奖励信号, 更新其内部的策略或价值函数。
- **3. 行动:** 根据当前状态和学习到的策略, 选择一个动作 (Action) 。



目标: 找到一个最优策略, 使得在长期交互中获得的累积奖励最大化。

什么是AI Agent?



AI Agent 本质上是构建一个在大模型的上层应用。如果把大模型比作人的大脑，虽然它拥有智能，却没有真正做事的实体，AI Agent就相当于人的眼睛、嘴巴、手、脚，以及各种人类可以利用的工具。

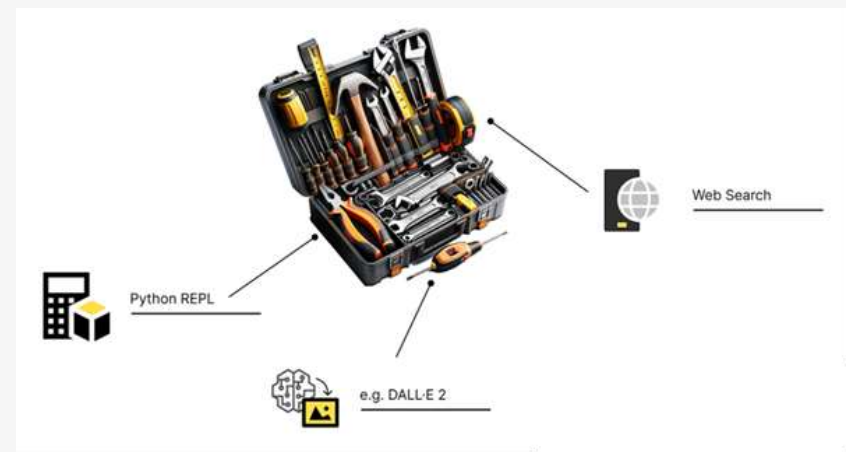
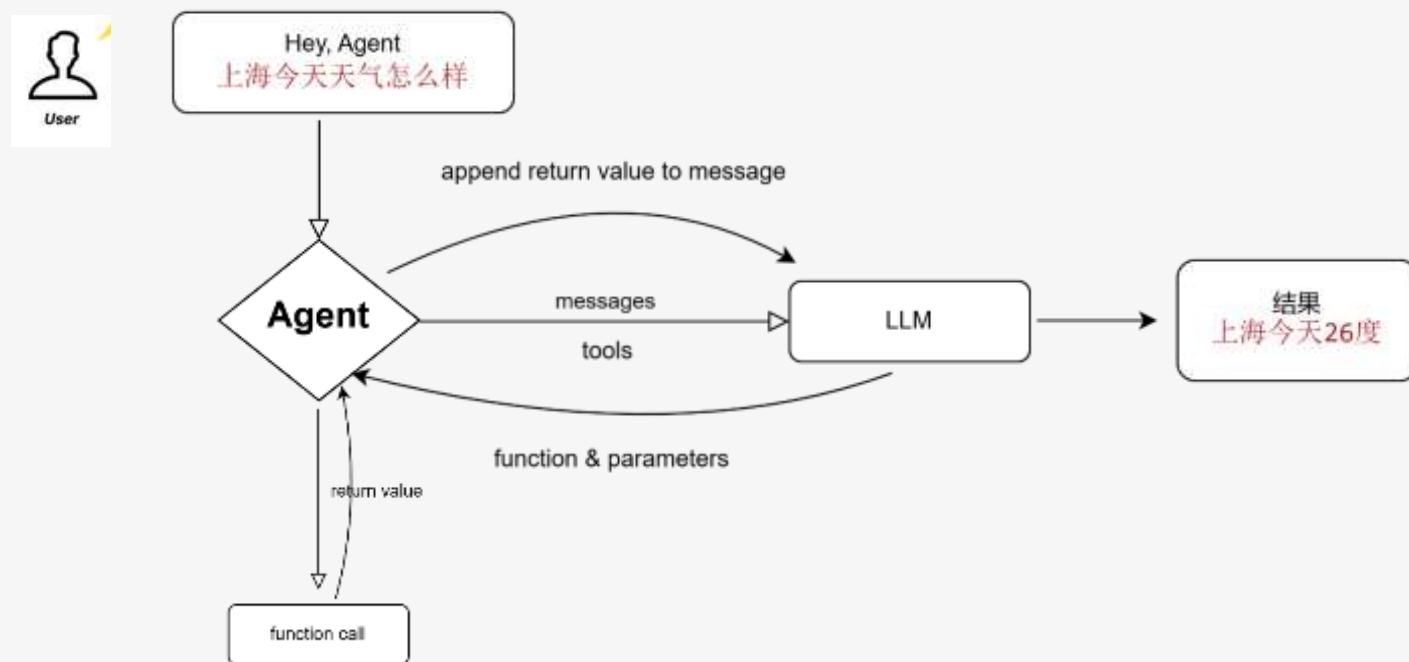
Agent的实现流程

感知 (Perception) → 规划 (Planning) → 行动 (Action)

感知是指Agent从环境中收集信息并从中提取相关知识。把能用的知识转换成提示，得到记忆。

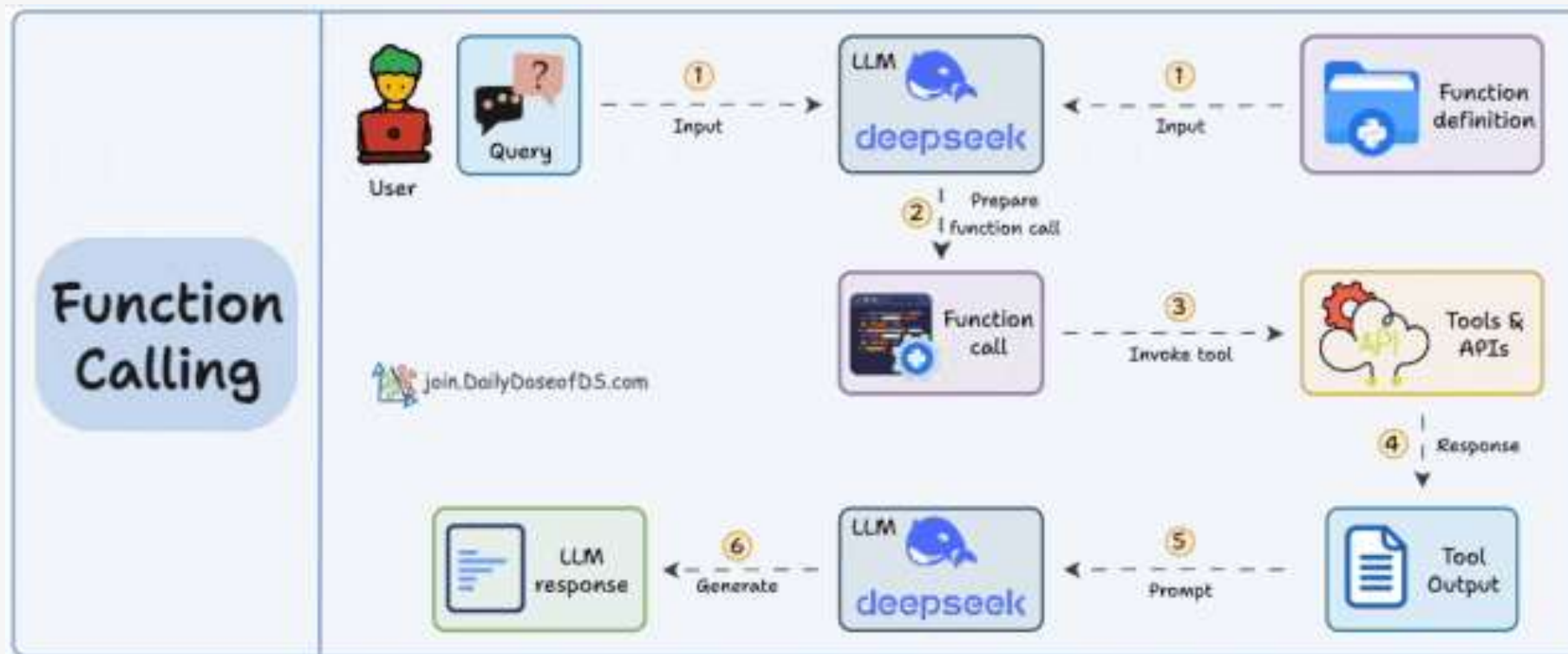
规划是指LLM为了某一目标而作出的决策过程，即总结看到的、感知到的、记忆中的东西。

行动包括一些API工具调用，如websearch、calculate等插件，得到部分结果，再反馈给LLM，进行下一步决策。



Function calling

Function Calling是通过自然语言让大模型智能地分析、调用外部函数或工具，以执行特定的操作。如去指定天气网站查询天气、文件读写、收发邮件等。这种功能可以解决信息实时性和数据局限性等问题。



JSON (JavaScript Object Notation)

JSON是一种轻量级的数据交换格式，易于人阅读和编写，同时也易于机器解析和生成。

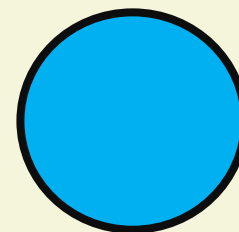
JSON数据是由键值对组成的。一个键值对就是一个属性“名称”和它的“值”。JSON对象被包含在花括号{}中，而JSON数组被包含在方括号[]中。

Json数组：

```
[  
  {"id": "1",  
   "type": "dialogue",  
   "content": [  
     {"user", "你好，能介绍一下人工智能吗？"},  
     {"assistant", "人工智能（AI）是一门研究如何使计算机  
系统能够执行通常需要人类智能才能完成的任务的学科"}]  
  },  
  ...  
]
```

```
TOOLS = [  
  {  
    "type": "function",  
    "function": {  
      "name": "add_two_numbers",  
      "description": "Adds two numbers and returns the result.",  
      "parameters": {  
        "type": "object",  
        "properties": {  
          "num_a": {  
            "type": "number",  
            "description": "The first number."  
          },  
          "num_b": {  
            "type": "number",  
            "description": "The second number."  
          }  
        },  
        "required": ["num_a", "num_b"],  
      }  
    }  
  }  
]
```

动手实践时间



Demo Time!

4. Function calling 查询天气

